

Índice.

1. Creando un servidor mínimo con Express.
2. Servicios REST.
3. Clientes.json.
4. GET: Envío de todo el contenido.
5. GET: Búsqueda por un campo.
6. PUT: Insertar un nuevo cliente.
7. Nombre de las rutas.
8. Apps que realizan peticiones http.
9. Ejercicio.

Express es un framework para node que nos facilita crear servidores web y microservicios con node. En esta introducción veremos las partes más básicas de express pues estudiarlo con profundidad excede la intención de este módulo. En concreto usaremos express para generar servicios REST.

1. Creando un servidor mínimo con Express.

Para crear un servidor mínimo vamos a crear una carpeta llamada **restexpress**, dentro de esta carpeta:

- Crear fichero index.js vacío por ahora.
- Generar el package.json con npm init
- Añadir el script "start" : "node index.js".
- Instalar express en esta app: npm install express.

Ya estamos en situación de crear un servidor básico con express, vamos a escribir en index.js:

```
const express = require('express');

let app = express(); //cargamos express

//definimos una ruta: http://localhost:8080/bienvenida
//y una respuesta a esa ruta: mensaje "Hola, bienvenido/a"
app.get('/bienvenida', (req, res) => {
  res.send('Hola, bienvenido/a');
});
//iniciamos el servicio en el puerto 8080
app.listen(8080);
```

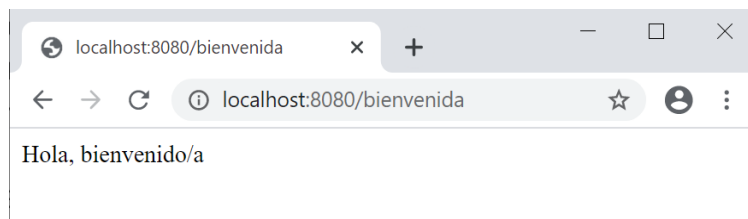
app.get, es un callback que se ejecutará cuando se reciba una petición (cuando un navegador o programa haga una petición a esta url: <http://localhost:8080/bienvenida>).

Como se puede ver el método **get** tiene dos parámetros: el objeto que contiene la petición (típicamente llamado **req**, abreviatura de **request**), y el objeto para emitir la respuesta (típicamente llamado **res**, abreviatura de **response**). Más adelante veremos qué otras cosas podemos hacer con estos objetos, pero de momento emplearemos la respuesta para enviar (**send**) texto al cliente que solicitó el servicio.

app.listen empieza a escuchar por el puerto establecido peticiones **http**.

Si pruebas a ejecutar la aplicación, verás que no finaliza. Hemos creado un pequeño servidor **Express** que queda a la espera de peticiones de los clientes. La manera más fácil de probarlo es escribir la siguiente ruta en un navegador:

<http://localhost:8080/bienvenida>



2. Servicios REST.

Intentaremos desarrollar un servidor sencillo que proporcione una API REST a los clientes que se conecten. REST son las siglas de REpresentational State Transfer, y designa un estilo de arquitectura de aplicaciones distribuidas, como las aplicaciones web. En un sistema REST, identificamos cada recurso a solicitar con una URI (identificador uniforme de recurso), y definimos un conjunto delimitado de comandos o métodos a realizar:

- **GET**: para obtener resultados de algún tipo (listados completos o filtrados por alguna condición)
- **POST**: para realizar inserciones o añadir elementos en un conjunto de datos
- **PUT**: para realizar modificaciones o actualizaciones del

conjunto de datos

- DELETE: para realizar borrados del conjunto de datos

Por lo tanto, identificando el recurso a solicitar y el comando a aplicarle, el servidor que ofrece esta API REST proporciona una respuesta a esa petición. Esta respuesta típicamente viene dada por un mensaje en formato JSON. Veremos cómo podemos identificar los diferentes tipos de comandos de nuestra API, y las URIs de los recursos a solicitar, para luego dar una respuesta en formato JSON ante cada petición.

3. Clientes.json.

Como la respuesta que queremos dar es en formato json, usaremos un fichero que ya utilizamos en el tema 4:

```
[
  {
    "dni": "11111111",
    "name": "cliente1",
    "phone": "91111111"
  },
  {
    "dni": "22222222",
    "name": "cliente2",
    "phone": "92222222"
  },
  {
    "dni": "33333333",
    "name": "cliente3",
    "phone": "93333333"
  }
]
```

4. GET: Envío de todo el contenido.

Vamos a crear una ruta para enviar a los clientes todo el contenido. Pero antes vamos a instalar un componente (el nombre técnico es middleware) que usará express para procesar las respuestas en formato json: `npm install body-parser`.

Para que pueda funcionar con express debemos escribir algunas líneas en `index.js`:

```
const express = require('express');
const bodyParser = require('body-parser');
const fs = require('fs'); //para acceder a los ficheros

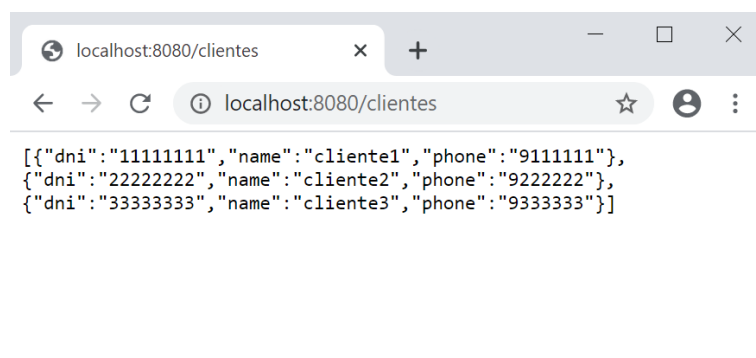
let app = express();
// Body-parser para procesar datos JSON desde el cuerpo de las peticiones
// POST/PUT
app.use(bodyParser.json());

//ruta de bienvenida
app.get('/bienvenida', (req, res) => {
  res.send('Hola, bienvenido/a');
});
```

Ahora crearemos una ruta para enviar el contenido de todo el fichero a los clientes:

```
// Servicio de listado por todos
app.get('/clientes', (req, res) => {
  //leer clientes del archivo
  let fichero = fs.readFileSync('./clientes.json');
  let clientes = JSON.parse(fichero);
  res.send(clientes);
});
```

Si ahora iniciamos el servidor y en un navegador vamos a <http://localhost:8080/clientes> :



En el código anterior hemos usado el método `get` de un instancia de `express`. En este método tenemos dos parámetros (como se menciona mas arriba):

- `req` (de request). El objeto `res` nos permite configurar lo que enviamos al cliente que hace la petición. En este caso enviamos el contenido de un fichero JSON.
- `res` (de response). El objeto `req` nos da información sobre la petición que ha hecho el cliente. En este ejemplo no

se usa, pero si lo haremos en otros.

5. GET: Búsqueda por un campo.

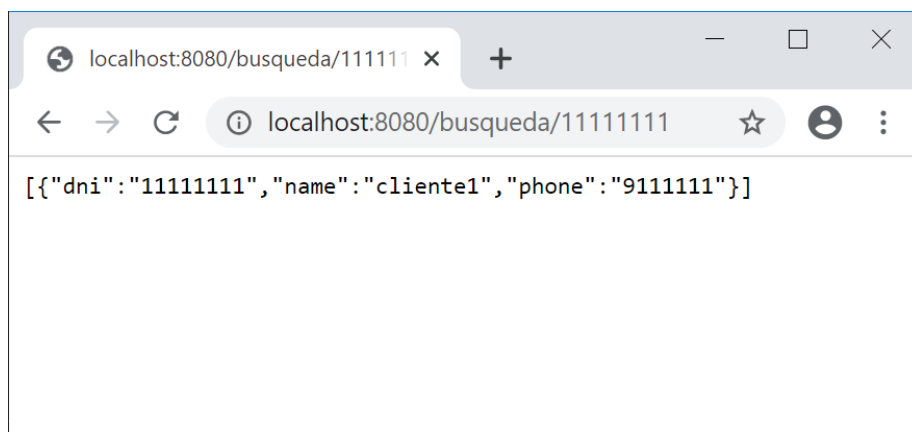
Para buscar por el campo dni para obtener un dni concreto podemos usar el código siguiente:

```
// Servicio de listado por dni
app.get('/clientes/:dni', (req, res) => {
  //leer clientes del archivo
  let fichero = fs.readFileSync('./clientes.json');
  let clientes = JSON.parse(fichero);
  //buscar cliente con dni en array clientes
  let clients = clientes.filter(client => client.dni == req.params.dni);

  if (clients.length != 0)
    res.send( clients );
  else
    res.send(«No se han encontrado clientes con ese DNI»);
});
```

En este caso la llamada a get contiene un parámetro dni, usaremos req.params para acceder al valor de ese parámetro.

Para probar esto:



6. PUT: Insertar un nuevo cliente.

Para incluir una ruta que inserte un nuevo cliente:

```
// Servicio para insertar clientes
app.post('/inserta', (req, res) => {
  try {
    //obtener el cliente dado con la petición post
    let nuevoCliente = req.body;
    //leer clientes del archivo
    let fichero = fs.readFileSync('./clientes.json');
```

```

    let clientes = JSON.parse(fichero);
    //añadir el nuevo cliente:
    clientes.push(nuevoCliente);
    //guardar el fichero completo:
    fs.writeFileSync('./clientes.json', JSON.stringify(clientes));
    res.send({ ok: true });
  }
  catch (err) {
    res.send({ ok: false });
  }
});

```

Suponemos que el objeto a insertar viene en formato JSON y lo recuperamos a través de req.body. Devolvemos un true o un false al programa que realice la petición para que sepa si fue bien o mal.

7. Nombre de las rutas.

Durante este ejemplo hemos nombrado las rutas así:

```

/bienvenida
/clientes
/clientes/:dni
/inserta

```

Se ha hecho para mejorar la comprensión del servidor express. Estos nombres son perfectamente válidos pero los tres últimos actúan sobre el mismo conjunto de datos: clientes.json. En estos casos se utiliza el mismo nombre de ruta, express los diferencia por el método con el que se accede a ellas (get, put, post, delete) y por los parámetros que lleven. Aplicando esto, el código completo de index.js queda mejor así:

```

const express = require('express');
const bodyParser = require('body-parser');
const fs = require('fs'); //para acceder a los ficheros

let app = express();
// Body-parser para procesar datos JSON
//desde el cuerpo de las peticiones POST/PUT
app.use(bodyParser.json());

//ruta de bienvenida
app.get('/bienvenida', (req, res) => {
  res.send('Hola, bienvenido/a');
});

// Servicio de listado por todos
app.get('/clientes', (req, res) => {

```

```

    //leer clientes del archivo
    let fichero = fs.readFileSync('./clientes.json');
    let clientes = JSON.parse(fichero);
    res.send(clientes);
  });

// Servicio de listado por dni
app.get('/clientes/:dni', (req, res) => {
  //leer clientes del archivo
  let fichero = fs.readFileSync('./clientes.json');
  let clientes = JSON.parse(fichero);
  //buscar cliente con dni en array clientes
  let clients = clientes.filter(client => client.dni == req.params.dni);

  if (clients != null)
    res.send( clients );
  else
    res.send({
      mensajeError: "No se han encontrado clientes con ese DNI"
    });
});

// Servicio para insertar clientes
app.post('/clientes', (req, res) => {
  try {
    //obtener el cliente dado con la petición post
    let nuevoCliente = req.body;
    //leer clientes del archivo
    let fichero = fs.readFileSync('./clientes.json');
    let clientes = JSON.parse(fichero);
    //añadir el nuevo cliente:
    clientes.push(nuevoCliente);
    //guardar el fichero completo:
    fs.writeFileSync('./clientes.json', JSON.stringify(clientes));
    res.send({ ok: true });
  }
  catch (err) {
    res.send({ ok: false });
  }
});

app.listen(8080);

```

8. Apps que realizan peticiones http.

En el punto 1 de este tema vimos como node incorpora un módulo para realizar peticiones http. Hay diversos módulos que se pueden instalar con rpm que nos facilitan esta tarea. En concreto ahora vamos a usar node-fetch (que da la misma funcionalidad que fetch en navegadores). Creamos un proyecto client donde se usará node-fetch para probar las rutas que hicimos en el proyecto restexpress arriba realizado:

- Crea una carpeta llamada **client**, dentro un archivo vacío app.js.
- Con npm init se genera el package.json, donde:

```
1 package.json > ...
2 {
3   "name": "client",
4   "version": "1.0.0",
5   "description": "",
6   "main": "app.js",
7   "scripts": {
8     "start": "node ."
9   },
10  "author": "",
11  "license": "ISC",
12  "type": "module",
13  "dependencies": {
14    "node-fetch": "^3.1.0"
15  }
```

- Instalamos node-fetch, desde la carpeta client: npm install node-fetch.

8.1. Módulos CommonJS y EcmaScript

Ahora debemos parar un poco para entender la opción `type` del `package.json`. En el tema 2 punto 2 se vio el uso de módulos en node. Existen dos formas de tratar con módulos: usando módulos CommonJS y usando módulos EcmaScripts (o ESM).

Los primeros son los que usa node por defecto y los que hemos ido utilizando durante todo el curso y son los explicados en el tema 2 punto 2. Sin embargo los ESM se están haciendo cada vez más importantes, tanto es así que la última versión de `node-fetch` sólo soporta la notación ESM.

Hay algunas diferencias entre los dos tipos de módulos, la más importante es `require` en `commonJs` e `import` en ESM. Para un mayor detalle sobre los módulos ESM:

<https://nodejs.org/api/esm.html>

Node soporta ambos tipos de módulos, CommonJS lo hace por defecto y ESM lo hace si añadimos la opción `type` del `package.json`.

8.2. App.js.

Ahora en `app.js` iremos realizando peticiones.

Probar la ruta `/bienvenida`

```
//const fetch = require('node-fetch'); no soportado pues usamos ESM
import fetch from 'node-fetch';
const recurso = "http://127.0.0.1:8080";

//Get para bienvenida:
fetch(recurso + '/bienvenida')
  .then(res => res.text())
  .then(body => console.log(body));
```

Con `fetch` se realiza la petición asíncrona del recurso. A esto se encadenan dos promesa `.then`, una para `res` (de response) para obtener los datos y otra con `body` para utilizar los datos

devueltos.

Si ejecutamos este proyecto (antes debemos tener en ejecución que da los servicios y que llamamos restexpress):

```
C:\Users\usuario\Documents\ProyectosNode\client>npm start

> client@1.0.0 start C:\Users\usuario\Documents\ProyectosNode\client
> node .

Hola, bienvenido/a

C:\Users\usuario\Documents\ProyectosNode\client>
```

Probar la ruta /clientes.

En este caso debemos obtener un fichero json:

```
//Get para todos los clientes:
fetch(recurso + '/clientes')
  .then(res => res.json())
  .then(json => console.log(json));
```

Por lo tanto en la primer then usamos el método json de res. Si ejecutamos:

```
C:\Users\usuario\Documents\ProyectosNode\client>npm start

> client@1.0.0 start C:\Users\usuario\Documents\ProyectosNode\client
> node .

Hola, bienvenido/a
[ { dni: '11111111', name: 'cliente1', phone: '91111111' },
  { dni: '22222222', name: 'cliente2', phone: '92222222' },
  { dni: '33333333', name: 'cliente3', phone: '93333333' } ]
```

Probar la ruta /clientes:dni

```
//Get para búsqueda por dni:
let dni = "11111111";
fetch(recurso + '/clientes/' + dni)
  .then(res => res.json())
  .then(json => console.log(json));
```

```
C:\Users\usuario\Documents\ProyectosNode\client>npm start

> client@1.0.0 start C:\Users\usuario\Documents\ProyectosNode\client
> node .

Hola, bienvenido/a
[ { dni: '11111111', name: 'cliente1', phone: '91111111' },
  { dni: '22222222', name: 'cliente2', phone: '92222222' },
  { dni: '33333333', name: 'cliente3', phone: '93333333' } ]
[ { dni: '11111111', name: 'cliente1', phone: '91111111' } ]
```

Probar la ruta post /clientes.

```
//nuevo objeto con el formato apropiado:
let nuevo = {
  "dni": "72345672",
  "name": "cliente",
  "phone": "91111111"
};
//post insertar cliente
fetch(recurso + '/inserta', {
  method: 'post',
  body: JSON.stringify(nuevo),
  headers: { 'Content-Type': 'application/json' },
})
.then(res => res.json())
.then(json => console.log(json));
```

```
C:\Users\usuario\Documents\ProyectosNode\client>npm start

> client@1.0.0 start C:\Users\usuario\Documents\ProyectosNode\client
> node .

Hola, bienvenido/a
[ { dni: '11111111', name: 'cliente1', phone: '91111111' },
  { dni: '22222222', name: 'cliente2', phone: '92222222' },
  { dni: '33333333', name: 'cliente3', phone: '93333333' } ]
[ { dni: '11111111', name: 'cliente1', phone: '91111111' } ]
{ ok: true }
```

Además en este caso podremos comprobar que se ha insertado en el fichero clientes.json un nuevo registro.

9. Ejercicio.

1. Define un servidor básico que responda por GET a estas dos URIs:

/fecha: el servidor enviará como respuesta al cliente la fecha y hora actuales. Puedes utilizar el tipo Date de Javascript Standard sin más:

https://developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Objetos_globales/Date

O también puedes usar la librería "moment", instalando npm install moment:

<https://momentjs.com/>

/usuario: el servidor enviará el login del usuario que entró al sistema. Necesitarás emplear la librería "os" del núcleo de Node:

<https://nodejs.org/api/os.html>

2. Crea un cliente para probar esas rutas.